



GitHub Trending Top 10

每日热门开源项目深度解读

2026-09-14

今日榜单

1	JustVugg/colibri	C	Run frontier MoE models on hardware you already own — pure C, zero deps, experts...	2天
2	alibaba/open-code-review	Go	Fast, efficient, battle-tested at Alibaba's scale. Hybrid architecture code revi...	NEW
3	multimodal-art-projection/YuE	Python	YuE2: frontier music generation with symbolic planning, zero-shot covers, and ag...	2天
4	debpalash/VoiceStudio	Python	VoiceStudio is the open-source, fully-local ElevenLabs alternative — voice cloni...	NEW
5	666ghj/MiroFish	Python	A Simple and Universal Swarm Intelligence Engine, Predicting Anything. 简洁通用的群体智...	NEW
6	Panniantong/Agent-Reach	Python	Give your AI agent eyes to see the entire internet. Read & search Twitter, Reddi...	NEW
7	asgeirtj/system_prompts_leaks	JavaScript	Extracted system prompts from Anthropic - Claude Fable 5.1, Opus 5, Claude Desig...	3天
8	rlaope/oh-my-hermes	Python	All in one plugin for Hermes Agent 🦂 the coding intelligence, a long-term memory...	NEW
9	localsend/localsend	Dart	An open-source cross-platform alternative to AirDrop	NEW
10	dani-garcia/vaultwarden	Rust	Unofficial Bitwarden compatible server written in Rust, formerly known as bitwar...	NEW

#1

C

连续 2 天

colibri

Run frontier MoE models on hardware you already own — pure C, zero deps, experts streamed from disk. Tiny engine, immense model. 

Stars **31,466** Forks **3,346** Today **+2,233**

<https://github.com/JustVugg/colibri>

项目简介：Colibri 是一个纯 C 编写、零引擎依赖的大模型推理引擎，目标是在消费级和异构硬件上运行 744B 到 2.8T 参数的 MoE（混合专家）前沿模型。它通过把显存、内存和磁盘视为统一的多级推理层级（AI memory multitiering），让普通人用自己已有的硬件"持有"而非"租用"大模型能力。

核心功能： - 支持九大模型家族（GLM-5.x、Kimi K3、DeepSeek V4、Qwen3.x、OLMoE 等），每个模型一个 C 文件，共用 coli chat / coli serve / coli web 前端。 - 专家权重从磁盘流式加载，按需在 VRAM/RAM/disk 三层间调度，实现"小引擎跑巨模型"。 - 内置 Web 仪表盘，实时可视化 token 指标、专家分层占用和路由热力（Brain/Atlas 页面把上万专家画成"活体皮层"和 3D 星系）。 - 提供 OpenAI 兼容的 serve 接口，便于接入现有应用。

适用场景：适合没有 hyperscaler 级算力、但想本地私有化运行超大 MoE 模型的开发者、研究者和小团队；也适合做推理系统研究的人——需要反复测量存储 I/O、内存层级、调度与投机解码等端到端性能的场景。

技术亮点： - 将存储/RAM/VRAM 作为单一推理层级统一编排，是典型的"以 I/O 换显存"的系统级思路。 - 明确承诺**不静默改变模型精度或路由语义**——快内存不足只降速，不偷偷改模型，这在工程伦理上很克制。 - 纯 C、零依赖、单文件模型实现，代码量小到足以让人"改得动"，把引擎本身当作开放研究平台而非黑盒产品。

上手建议：先从官网和 release 页下载最新版，用 ./coli chat 跑通一个小模型（如 OLMoE 7B）验证环境，再逐步尝试更大模型；想贡献的话，README 里的专家图谱（Atlas）和模型 roster 是很好的切入点，Discord 社区和 GitHub issue 是主要协作渠道。注意项目明确"不承诺速度 SLA"，适合愿意折腾、关注可复现测量的用户。

#2

Go

NEW

open-code-review

Fast, efficient, battle-tested at Alibaba's scale. Hybrid architecture code review tool: deterministic pipelines + LLM Agent, precise line-level comments, built-in multi-language ruleset (NPE, thread-safety, XSS, SQL injection), OpenAI & Anthropic compatible.

Stars **24,985** Forks **1,827** Today **+1,796**

<https://github.com/alibaba/open-code-review>

项目简介：Open Code Review 是阿里巴巴开源的一款 AI 代码审查 CLI 工具，脱胎于阿里内部服务数万名开发者、识别数百万代码缺陷的官方 AI 评审助手。它读取 Git diff 后交给可配置的 LLM Agent 分析，输出行级精度的结构化评审意见，解决传统人工评审耗时、通用 AI 工具噪音大的痛点。

核心功能：一是基于 Git diff 的智能评审，Agent 可读取完整文件、检索代码库、交叉查看其他变更文件以获取上下文；二是 ocr_scan 支持对无 diff 的整文件扫描，适合审计陌生代码库；三是内置多语言规则集（NPE、线程安全、XSS、SQL 注入等）；四是兼容 OpenAI 与 Anthropic 接口，并支持 Claude Code、Codex、Cursor 等主流 Agent。

适用场景：适合需要在高频 PR 流程中自动拦截缺陷的研发团队，尤其是中大型项目、多语言代码库的日常 CI/CD 集成。也适合个人开发者审计陌生仓库，或安全团队做静态缺陷排查。

技术亮点：采用"确定性流水线 + LLM Agent"的混合架构，兼顾规则引擎的稳定性与 Agent 的推理深度。在自建 AACR-Bench（50 仓库、200 PR、10 语言、1505 条人工标注）上，同模型下 F1 与精确率显著优于通用 Agent，Token 消耗仅约 1/9，且主动牺牲召回率换取低误报——这是针对"评审噪音"的刻意取舍。

上手建议：最简路径是配置一个模型端点后通过 npm 包 @alibaba-group/open-code-review 直接运行，无需复杂部署。想深入可先阅读官网文档与 DeepWiki，贡献方向可关注多语言规则集扩展、Agent 工具能力增强，或基于 AACR-Bench 数据集做评测对比。

#3

Python

连续 2 天

YuE

YuE2: frontier music generation with symbolic planning, zero-shot covers, and agentic music editing.

Stars **8,176** Forks **895** Today **+578**

<https://github.com/multimodal-art-projection/YuE>

项目简介：YuE2 是由港科大、M·A·P、斯坦福等多家机构联合开发的音乐生成模型，目标是在开源领域实现接近 Suno 级别的整曲生成质量。它解决的核心问题是：让 AI 音乐生成从"黑盒出音频"变成"可读、可改、可编辑的白盒创作流程"。

核心功能：

- **符号化规划生成：**输入歌词和风格提示后，先生成旋律与和弦的显式乐谱计划，再渲染为带人声和伴奏的完整歌曲。
- **零样本翻唱 (Cover)：**将一段转录后的录音重新演绎成全新风格，无需针对目标风格额外训练。
- **智能体编辑 (Agentic Editing)：**通过对话方式对乐谱、编曲、歌词进行多轮修改，逐步迭代出满意版本 (Demo 中展示了 9 步 14 版的编辑过程)。
- **分阶段 Python API：**暴露 `plan()` → `generate_semantic()` → `synthesize()` → `decode()` 的流水线，便于精细控制。

适用场景：适合 AI 音乐研究者、独立音乐人、以及希望将音乐生成能力集成到产品中的开发者。典型场景包括快速创作 Demo 歌曲、为已有旋律生成新编曲、以及通过对话式交互进行音乐二次创作。

技术亮点：采用 AR-NAR 混合的 Mixture-of-Transformers 架构，自回归预测乐谱与语义 token，再用 flow matching 生成声学 latent，最终由 VAE 解码为 48kHz 立体声。关键创新在于"符号规划"这一中间层——它让旋律和和弦成为人和智能体都能检查和编辑的显式控制量，而非不可见的隐变量。在 WildSongBench 上 best-of-8 取得 6.9632 的平均分，是目前评测中的最高值。

上手建议：环境要求为 Linux + Python 3.12 + 支持 BF16 的 24GB 显存 NVIDIA GPU，直接 `pip install` 后运行 `examples/generate.py` 即可产出第一首歌。建议先跑通示例、查看输出目录中保留的乐谱与 latent 文件，再阅读 `docs/generation.md` 了解精确规划复用与解码器选择；贡献方面可从 benchmarks 复现或 agent skill 扩展入手。

#4

Python

NEW

VoiceStudio

VoiceStudio is the open-source, fully-local ElevenLabs alternative — voice cloning, voice design, video dubbing, dictation, transcription & audiobook creation in 646 languages.

Stars **28,633** Forks **3,505** Today **+2,774**

<https://github.com/debpalash/VoiceStudio>

项目简介：VoiceStudio 是一个完全本地运行的开源语音工作室，定位为 ElevenLabs 的替代方案，把语音克隆、配音、听写、转录和有声书制作整合到一个桌面应用中。它解决的核心问题是：用户无需账号、API Key、订阅或用量计费，就能在自己的硬件上完成专业级语音处理，且数据默认不出本机。

核心功能：一是语音克隆与设计，支持"从音频克隆""按设计生成""语音转换"三种工作流；二是视频配音，支持文件上传或 URL 导入，并保留历史项目可随时重开；三是听写与转录，覆盖长音频和有声书制作；四是多引擎切换，内置 16 个 TTS 引擎和 11 个 ASR 引擎，可通过快捷键快速切换；五是本地 API 服务，提供 REST/SSE/WebSocket 以及兼容 OpenAI 的音频接口和 MCP Server。

适用场景：适合对隐私敏感的内容创作者、播客和有声书制作者、需要批量生成语音的开发者，以及想用视频配音做多语言分发的团队。典型场景包括：用自己或授权的声音克隆出旁白、把一段视频自动配成 646 种语言中的任意一种、在本地批量生成语音素材而不产生云端费用。

技术亮点：跨平台覆盖 macOS (Apple Silicon)、Windows、Linux 和 Docker，计算后端同时支持 CUDA、Apple MPS/MLX、ROCm 和纯 CPU，并可选远程 worker 扩展算力。工程上采用多引擎抽象层，让 TTS/ASR 引擎可以热切换和卸载，同时把桌面应用、本地 API 和 MCP Server 三种接口统一在同一套架构下。目前正在进行 Electron 重写，说明团队在主动重构桌面端体验。

上手建议：普通用户直接从最新 release 下载对应平台安装包即可，注意项目处于活跃 beta 阶段，稳定工作请用 release 而非 main 分支。开发者可以先阅读 README 中的 Architecture 和 API 章节，通过本地 REST 接口做集成测试；想贡献代码的话，由于 Electron 重写正在进行，官方明确建议暂时不要提交桌面应用相关的 issue 和 PR，可以从引擎适配、文档或非桌面模块入手。遇到问题走 GitHub Issues，社区交流在 Discord。

#5

Python

NEW

MiroFish

A Simple and Universal Swarm Intelligence Engine, Predicting Anything. 简洁通用的群体智能引擎，预测万物

Stars **72,960** Forks **11,276** Today **+524**

<https://github.com/666ghj/MiroFish>

项目简介：MiroFish 是一个基于多智能体（Multi-Agent）技术的通用群体智能预测引擎，目标是用 AI 模拟现实世界的演化来"预测万物"。它解决的是传统预测方法难以捕捉复杂社会系统中个体互动与集体涌现行为的问题，通过构建"平行数字世界"来推演未来走向。

核心功能：一是从真实世界的种子信息（新闻、政策草案、金融信号等）自动构建高保真数字沙盘；二是让成千上万个具备独立人格、长期记忆和行为逻辑的智能体自由交互、社会演化；三是支持"上帝视角"动态注入变量，精确推演不同条件下的未来轨迹；四是输出详细预测报告和可深度交互的数字世界；五是支持上传任意素材（数据分析报告甚至小说故事）并用自然语言描述预测需求。

适用场景：宏观层面适合政策制定者、公关团队做零风险的决策预演，比如舆情事件走向模拟（README 中演示了武汉大学舆情案例）；微观层面适合个人创作者做创意探索，比如推演《红楼梦》遗失的结局。任何需要"如果……会怎样"推演的场景都可以尝试。

技术亮点：核心在于大规模多智能体系统的构建——每个 Agent 拥有独立人格、长期记忆和行为逻辑，能在数字空间中自发产生社会演化与集体涌现，而非简单的规则推演。项目背靠盛大（Shanda）生态，提供 Docker 部署、在线 Demo 和 DeepWiki 文档支持，工程化程度较高。

上手建议：先访问在线 Demo（mirofish-live-demo）体验舆情模拟和《红楼梦》结局推演两个官方案例，建立直观认知；然后参考 README-ZH 中文文档和 DeepWiki 了解架构，用 Docker 快速搭建本地环境。想贡献的话，建议从自己熟悉的领域（如金融、舆情、文学）构造种子素材跑通完整流程，再针对 Agent 行为逻辑或预测报告质量提出改进。

#6

Python

NEW

Agent-Reach

Give your AI agent eyes to see the entire internet. Read & search Twitter, Reddit, YouTube, GitHub, Bilibili, XiaoHongShu — one CLI, zero API fees.

Stars **80,973** Forks **7,051** Today **+640**

<https://github.com/Panniantong/Agent-Reach>

项目简介：Agent Reach 是一个为 AI Agent 提供“上网能力”的 CLI 工具，用一句话就能让 Claude Code、Cursor 等 Agent 访问 Twitter、Reddit、YouTube、GitHub、B站、小红书等平台。它解决的核心痛点是：Agent 能写代码却“看不见互联网”，而各平台的 API 付费、风控封锁、登录限制让个人开发者接入成本极高。项目把选型、安装、诊断全部打包好，用户无需操心接入方式的迭代。

核心功能：

- 多平台统一接入：**一个 CLI 覆盖网页阅读、YouTube 字幕提取、RSS、全网搜索、GitHub、Twitter、Reddit、B站、小红书等，装好即用或简单配置后解锁。
- 零 API 费用：**所有工具开源、API 免费，唯一可能的花费是服务器代理（约 \$1/月），本地使用完全免费。
- 自动故障切换：**每个平台采用“首选 + 备选”多后端路由，某个接入方式失效时自动切换，用户无感知（如 B站风控封死 yt-dlp 后切换 bili-cli）。
- 自带诊断命令：**agent-reach doctor 一条命令检查各平台连通性并给出修复建议。
- 一句话安装/更新：**把安装文档链接丢给 Agent，几分钟完成部署，更新同样一句话搞定。

适用场景：

- **AI Agent 开发者：**想让自己的 Agent 具备实时互联网信息获取能力，用于调研、舆情监控、内容总结等。
- **个人研究者/内容创作者：**需要跨平台搜集信息（如查 Twitter 评价、Reddit 踩坑、小红书口碑、B站教程总结），但不想逐个折腾 API 和爬虫。
- **自动化 workflows 搭建者：**将 Agent Reach 作为信息采集层，接入 OpenClaw、Cursor、Windsurf 等任意能跑命令行的 Agent。

技术亮点：

- **多后端路由架构：**每个平台维护“首选 + 备选”接入方案，平台风控变化时只需替换后端，用户接口不变，这是项目宣称“接入方式会换代，你不用操心”的底层保障。
- **Cookie 本地化与隐私设计：**登录态 Cookie 只存本地、不上传，代码完全开源可审查，在需要登录的平台（如小红书）和隐私安全之间取得平衡。
- **Agent 原生交互：**安装和更新都通过“把文档链接发给 Agent”完成，而非传统 pip install，贴合 AI Agent 时代的工具分发方式。
- **MCP 自动配置：**全网搜索通过 MCP 接入，免费无需 Key，降低了语义搜索的接入门槛。

上手建议：

- **使用者：**直接把 README 中的安装文档链接发给你的 Agent（如 Claude Code），几分钟后即可使用；装好后先跑 `agent-reach doctor` 确认各平台状态，再按需配置 Twitter、小红书等需要登录的平台。
- **贡献者：**项目采用 MIT 协议、Python 3.10+，可从 `docs/install.md` 和 `agent-reach doctor` 的实现入手，理解多后端路由机制；平台风控变化频繁，贡献新的备选后端或修复失效接入是最高价值的参与方式。

#7

JavaScript

连续 3 天

system_prompts_leaks

Extracted system prompts from Anthropic - Claude Fable 5.1, Opus 5, Claude Design, Claude Code. OpenAI - ChatGPT GPT-6-Astra, Codex. Google - Gemini 3.8 Flash, 3.1 Pro, Antigravity. xAI - Grok, Grok Bot, Cursor, Kimi and more! Updated regularly.

Stars **66,581** Forks **10,852** Today **+770**

https://github.com/asgeirtj/system_prompts_leaks

项目简介：这是一个系统提示词（System Prompt）泄露合集仓库，收集了各大 AI 产品在用户输入第一条消息之前收到的隐藏指令。它解决的是 AI 黑箱问题——让外界能一窥 ChatGPT、Claude、Gemini、Grok 等模型"出厂设定"的行为规则和约束。

核心功能：一是按厂商和模型分类归档系统提示词，覆盖 Anthropic、OpenAI、Google、xAI、Meta、Perplexity、Kimi 等主流玩家；二是持续追踪更新，README 顶部维护"最新变更表"，标注具体日期和文件链接；三是记录同一产品的不同变体，比如 Claude Code 的 headless 模式、ChatGPT Work 本地版、各代模型迭代版本；四是附带部分产品的延伸资产，如 Claude Design 的 53 个工具、22 个技能和 10 个起始组件。

适用场景：AI 研究者、提示词工程师、产品经理、安全合规人员，以及想理解"AI 为什么这样回答"的开发者。典型用途包括对比不同厂商的行为约束设计、复现官方产品行为、审计 AI 的偏见或安全边界、为自研 Agent 借鉴提示词写法。

技术亮点：仓库本身以 Markdown 文件为主、用 JavaScript 辅助，工程上并不复杂，真正的价值在于"逐字捕获"（verbatim）的采集方式和持续维护的时效性——连华盛顿邮报和 CEPS 都基于它的数据做报道和可视化仪表盘，说明其数据被当作一手信源使用。分类维度也很细，按厂商、产品线、模型版本、部署形态（web/headless/本地）多层组织。

上手建议：直接浏览 README 的目录表，从你关心的厂商文件夹（如 Anthropic/、OpenAI/、Codex/）进入，挑最新日期的文件阅读即可。想贡献的话，README 明确标注 PRs Welcome，可以提交新捕获的提示词或修正已有内容；注意保持"逐字转录"的准确性，并附上来源和版本信息，这是这个仓库最核心的信任基础。

#8

Python

NEW

oh-my-hermes

All in one plugin for Hermes Agent ↑ the coding intelligence, a long-term memory system and model optimized workflow packages

Stars **1,888** Forks **153** Today **+52**

<https://github.com/rlaope/oh-my-hermes>

项目简介： oh-my-hermes (OMH) 是 Nous Research 的 Hermes Agent 之上的一层"操作层"插件，定位类似 oh-my-zsh 之于 zsh。它不替换 Hermes，而是在保持其自然语言交互方式的前提下，为 Agent 补上规划、研究、编码交接、运维和长期项目记忆等专业能力，并明确划定"证据边界"，让 Agent 如实记录到底做了什么。

核心功能： - **长期记忆系统：** 跨会话保存项目上下文，让 Agent 记住历史决策和项目状态。 - **模型优化的工作流包：** 针对不同任务类型（规划、研究、编码、运维）预置工作流。 - **证据边界机制：** 明确区分"声称完成"和"实际完成"，避免 Agent 虚报结果。 - **原生技能编排：** 把 Hermes 原生 skills 作为能力单元，纳入统一的治理路径中调度。 - **一键安装脚本：** 支持 macOS/Linux (curl) 和 Windows (PowerShell)，安装即用。

适用场景： 主要面向重度使用 Hermes Agent 进行日常开发或研究的用户，尤其是需要跨会话保持项目记忆、希望 Agent 输出更可追溯的开发者。也适合把 Agent 用于多阶段任务（先规划、再研究、再编码、再交接）的团队场景。

技术亮点： 设计上强调"不隐藏编码执行器"——OMH 自身不做代码执行，而是作为编排层决定用哪个 workflow、过哪些 evidence gate，再调用 Hermes 原生技能。这种"操作层与执行层分离"的架构，让能力可插拔、边界可审计，比把一切塞进一个黑盒 Agent 更可控。项目本身由 AI Agent (Firen、Killua) 参与协作开发，也是一个小亮点。

上手建议： 先阅读官方文档 (docs/README.md) 和安装说明 (docs/INSTALLATION.md)，用一行 curl 命令完成安装后跑通默认工作流。想深入的话可看 CAPABILITIES.md 了解能力清单和 CAPABILITY_IMPACT.md 了解各能力的影响范围。贡献方面，项目有 Discord 社区和 X (@rlaope) 更新渠道，建议先加入社区了解当前 roadmap 再提 PR。

#9

Dart

NEW

localsend

An open-source cross-platform alternative to AirDrop

Stars **91,105** Forks **5,087** Today **+213**

<https://github.com/localsend/localsend>

项目简介：LocalSend 是一款开源的跨平台本地文件传输工具，被广泛视为 AirDrop 的替代方案。它解决的核心问题是：在没有互联网、没有第三方服务器的情况下，让不同操作系统的设备之间也能快速、安全地互传文件和消息。

核心功能：一是跨平台互传，覆盖 Windows、macOS、Linux、Android、iOS 和 Fire OS；二是无需联网，完全依赖局域网通信；三是采用 HTTPS 加密和 REST API 实现安全传输；四是支持发送文件和文字消息；五是提供命令行接口，方便自动化或脚本调用。

适用场景：适合同一 Wi-Fi 下手机与电脑互传照片、文档，或在没有外网的办公、教学、会议环境中快速分享文件。对隐私敏感、不想把文件上传到云端的用户尤其友好，也适合需要在多设备间频繁交换资料的开发者和运维人员。

技术亮点：项目用 Dart 编写，借助 Flutter 实现一套代码多端运行；传输层基于 REST API + HTTPS，既保证加密又便于跨平台实现。它不依赖中心服务器，设备通过局域网自动发现彼此，架构轻量，避免了云服务常见的隐私和延迟问题。

上手建议：普通用户可直接从应用商店或包管理器安装，推荐优先使用商店版本以便更新。想参与贡献的话，可以从翻译（项目使用 Weblate）、修 bug 或改进功能入手；开发者则可参考 README 中各平台的构建说明，从对应平台的编译流程开始熟悉代码。

#10

Rust

NEW

vaultwarden

Unofficial Bitwarden compatible server written in Rust, formerly known as bitwarden_rs

Stars **67,451** Forks **3,206** Today **+110**

<https://github.com/dani-garcia/vaultwarden>

项目简介：Vaultwarden 是 Bitwarden 官方密码管理服务的非官方轻量级服务端实现，使用 Rust 编写，前身为 bitwarden_rs。它解决了官方服务端资源占用高、自托管门槛高的问题，让个人或小团队能在低配服务器上运行一个与官方客户端完全兼容的密码管理后端。

核心功能：完整实现 Bitwarden 客户端 API，支持个人保险库、Send 阅后即焚分享、附件存储、网站图标获取、个人 API Key 以及组织/集合/密码共享等团队协作功能。基本覆盖了官方服务端的绝大多数日常使用场景。

适用场景：适合注重隐私、希望自行托管密码库的个人用户、家庭或小型团队，尤其是那些拥有一台低配 VPS 或树莓派、不想为官方服务端付出高昂资源成本的场景。也适合对数据主权有要求、不希望密码数据存放在第三方云端的用户。

技术亮点：用 Rust 重写带来了极低的内存占用和优秀的性能，相比官方基于 .NET 的服务端在资源消耗上有数量级优势。同时它严格兼容官方 Bitwarden 客户端协议，用户无需更换任何客户端即可无缝切换，这是其能获得 6.7 万星标的关键原因。

上手建议：最简单的方式是通过 Docker 或 ghcr.io 镜像一键部署，官方文档和社区 Discourse 论坛提供了详细的配置说明。如果想贡献代码，建议先从阅读 README 中的构建指南和参与 GitHub Discussions 入手，该项目使用 AGPL-3.0 协议，社区活跃且对新手相对友好。

数据来源: github.com/trending

报告日期: 2026-09-14

由 DeepSeek AI 提供智能分析 | 自动生成